

Semântica Formal

João Eduardo Montandon
DCC024

O que esse programa faz? 🤔

```
int x = 1;
x += (x = 2);
```

Esses programas são iguais? 🤔

```
public class T {
    public static void main(String args[]) {
        int x = 1;
        x += (x = 2);
    }
}
```

```
#include <stdio.h>
int main() {
    int x = 1;
    x += (x = 2);
}
```

Relembrando Sintaxe

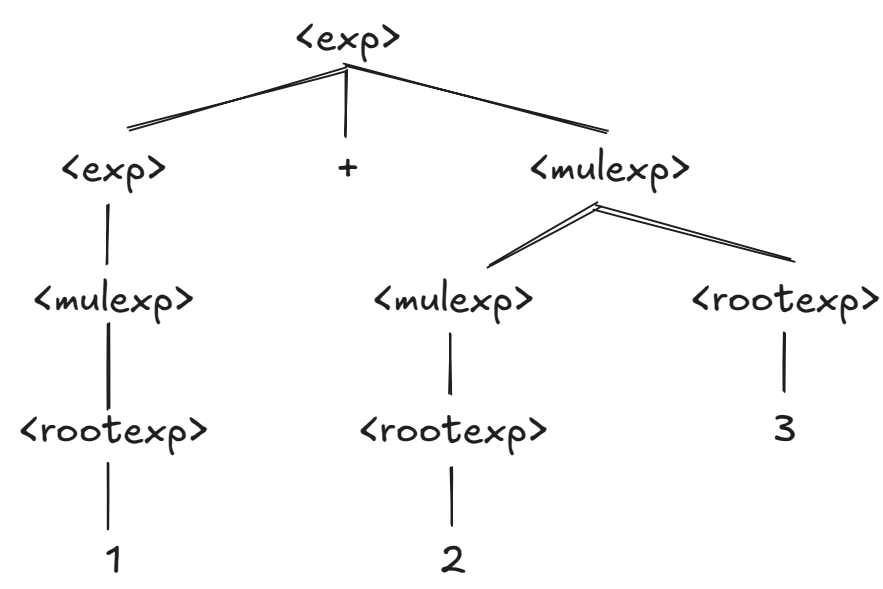
As aulas [02](#) e [03](#) nos mostraram como descrever a sintaxe de uma linguagem.

- **BNF:** gramática para representar estrutura da linguagem.
- **AST:** versão compacta da BNF.

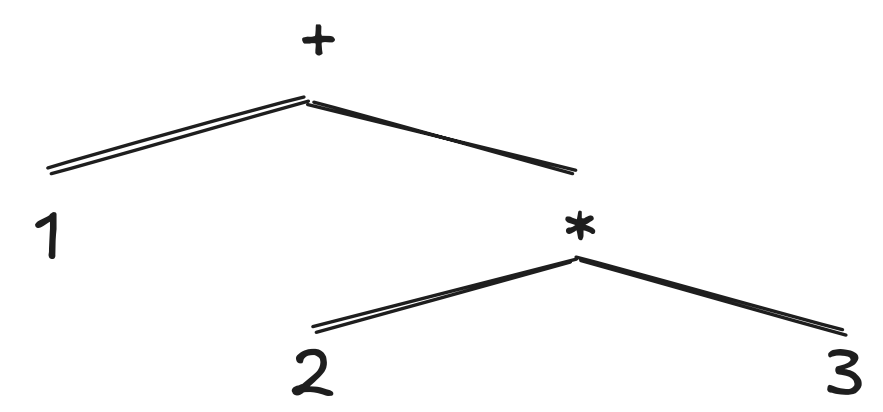
Qual seria a gramática para expressões numéricas simples?

```
<exp> ::= <exp> + <mulexp> | <mulexp>
<mulexp> ::= <mulexp> * <rootexp> | <rootexp>
<rootexp> ::= ( <exp> ) | <constant>
```

Qual seria a árvore de *parsing* para `1 + 2 * 3`?



E sua AST?



Agora vem o restante da história, "executar" essa representação.

Semântica

Se a sintaxe define a estrutura da linguagem, a **semântica define seu significado**.

- Entrada ➡ AST do programa.
- Saída ➡ "Valor" do programa.

Como definir o significado de uma linguagem? 🤔

Semântica Natural

Uma solução: **interpretar** a AST gerada.

```
<ast> ::= plus(<ast>, <ast>)
        | times(<ast>, <ast>)
        | const(<const>)
```

Iremos criar em Java? ML? ... Prolog! 🧐

Interpretador Em Prolog

- Linguagem 01: expressões.
- Linguagem 02: variáveis.

3. Linguagem 03: funções.

Linguagem 01

1 + 2 * 3



plus(const(1), times(const(2),const(3)))

```
evall(const(X), X).

evall(plus(X,Y), R) :-
    evall(X, RX),
    evall(Y, RY),
    R is RX + RY.

evall(times(X,Y), R) :-
    evall(X, RX),
    evall(Y, RY),
    R is RX * RY.
```

```
?- evall(const(2), X).
X = 2.

?- evall(plus(const(1),const(2)),R).
R = 3.

?- evall(times(const(4),const(2)),R).
R = 8.

?- evall(plus(const(1),times(const(2),const(3))),R).
R = 7.
```

Para Refletir

- Qual o valor de uma constante?
 - evall(const(X), X).
- Quais restrições são impostas a X? 🤖

```
?- val1(const(2147483647), X).
X = 2147483647.

?- val1(const(2147483648), X).
X = 2.14748e+009
```

Se você define uma semântica a partir de um interpretador, sua **semântica absorve o comportamento** deste interpretador. Cuidado!

Semântica Natural

- Definir a semântica consiste em mapear ASTs para valores.
 - times(const(2), const(3)) → 6.
- Semântica Natural permite definir essa relação **independente de linguagem**.

- $A \rightarrow v$ ➡ AST A foi avaliada para o valor v .
- $R = \frac{Cond}{Res}$ ➡ Dada as condições $Cond$, a regra R produz a conclusão Res .

$$\frac{E_1 \rightarrow v_1 \quad E_2 \rightarrow v_2}{times(E_1, E_2) \rightarrow v_1 \times v_2}$$



```
evall(times(X,Y), R) :-  
    evall(X, RX), evall(Y, RY), % condições  
    R is RX * RY. % conclusão
```

$$\frac{E_1 \rightarrow v_1 \quad E_2 \rightarrow v_2}{plus(E_1, E_2) \rightarrow v_1 + v_2}$$

$$\frac{E_1 \rightarrow v_1 \quad E_2 \rightarrow v_2}{times(E_1, E_2) \rightarrow v_1 \times v_2}$$

$$const(n) \rightarrow eval(n)$$

Como definir a semântica de expressões condicionais? 🤔

$$\frac{E_1 \rightarrow true \quad E_2 \rightarrow v_2}{if(E_1, E_2, E_3) \rightarrow v_2}$$

$$\frac{E_1 \rightarrow false \quad E_3 \rightarrow v_2}{if(E_1, E_2, E_3) \rightarrow v_2}$$

Linguagem 02

Como representar `let val y = 1 in y*y end`? 🤔

Gramática

```
<exp> ::= <exp> + <mulexp> | <mulexp>  
<mulexp> ::= <mulexp> * <rootexp> | <rootexp>  
<varexp> ::= let val <variable> = <exp> in <exp> end  
<rootexp> ::= <varexp> | ( <exp> ) | <constant>
```

AST

```
<ast> ::= plus(<ast>, <ast>)  
        | times(<ast>, <ast>)  
        | const(<const>)  
        | let(<var>, <ast>, <ast>)  
        | var(<name>, <const>)
```

Interpretador

Uma variável **aponta** para um valor, como representá-la? 🤔

- Contexto ➡ lista de tuplas, contendo as variáveis do programa.

- Tupla ➡ `bind(Variable, Value)`.

Exemplos:

- `y = 3` $\Rightarrow$ `[bind(y, 3)]`
- `x = 2 / y = 3` $\Rightarrow$ `[bind(x,2), bind(y,3)]`

Predicado `lookup`: busca variável em `Context`.

```
lookup(Variable,[bind(Variable, Value)|_], Value) :- !.  
lookup(Variable, [_|T], Value) :- lookup(Variable, T, Value).
```

Agora, devemos disponibilizar o Contexto para as regras.

```
evall(const(X), _, X).  
  
evall(plus(X,Y), C, R) :-  
    evall(X, C, RX),  
    evall(Y, C, RY),  
    R is RX + RY.  
  
evall(times(X,Y), C, R) :-  
    evall(X, C, RX),  
    evall(Y, C, RY),  
    R is RX * RY.
```

Por fim, implementamos as regras para `var` e `let`.

```
evall(var(X), C, R) :- lookup(X, C, R).  
  
evall(let(V, E1, E2), C, RFinal) :-  
    evall(E1, C, RTemp), % Obtém valor de E1 (valor de V)  
    evall(E2, [bind(V, RTemp)|C], RFinal). % "Insere" V em C e processa E2
```

```
% let x = 3 in 2 + x end  
?- evall(let(x,const(3),plus(const(2),var(x))),[],R).  
R = 5  
  
% let y = 3 in y*y end  
?- evall(let(y,const(3),times(var(y),var(y))),[],R).  
R = 9.  
  
% let val y = 3 in  
%     let val x = y * y in  
%         x * x  
%     end  
% end  
?- evall(let(y,const(3),  
|         let(x,times(var(y),var(y)),  
|             times(var(x),var(x))),  
|         nil, R).  
R = 81.
```

$$\frac{\langle E_1, C \rangle \rightarrow v_1 \quad \langle E_2, C \rangle \rightarrow v_2}{\langle plus(E_1, E_2), C \rangle \rightarrow v_1 + v_2}$$
$$\frac{\langle E_1, C \rangle \rightarrow v_1 \quad \langle E_2, C \rangle \rightarrow v_2}{\langle times(E_1, E_2), C \rangle \rightarrow v_1 \times v_2}$$
$$\langle const(n), C \rangle \rightarrow eval(n)$$

$$\langle var(n), C \rangle \rightarrow lookup(n, C)$$

$$\frac{\langle E_1, C \rangle \rightarrow v_1 \quad \langle E_2, bind(x, v_1) :: C \rangle \rightarrow v_2}{\langle let(x, E_1, E_2), C \rangle \rightarrow v_2}$$

Erros

```
let val a = 1 in b end
```

- O que há de errado?
- Qual o significado do seu erro? 🤔
- Como corrigir? 😞 😞

A correção vai depender de quando esse significado será processado.

- Semântica estática: verificada em tempo de **compilação**.
- Semântica dinâmica: verificada em tempo de **execução**.

Linguagem 03

Como adicionar funções a nossa gramática?

1. Definição da função.
2. Aplicação da função.

Gramática

```
<exp> ::= fn <variable> => <exp> | <addexp>
<addexp> ::= <exp> + <mulexp> | <mulexp>
<mulexp> ::= <mulexp> * <funexp> | <funexp>
<funexp> ::= <funexp> <rootexp> | <rootexp>
<varexp> ::= let val <variable> = <exp> in <exp> end
<rootexp> ::= <varexp> | ( <exp> ) | <constant>
```

- Definição da função possui menor precedência.
- Aplicação da função possui maior precedência.

```
(fn x => x * x) 3
```

AST


```
<ast> ::= plus(<ast>, <ast>)
        | times(<ast>, <ast>)
        | const(<const>)
        | let(<var>, <ast>, <ast>)
        | var(<name>, <const>)
        | fn(<param>, <body>)
        | apply(<fn>, <args>)
```

Duas novas regras:

- `apply(Function, Args)` aplica `Function` com os `Args`.
- `fn(Param, Body)` define a função com o parâmetro `Param` e `Body`.

```
(fn x => x * x) 3
```



```
apply(fn(x,times(var(x),var(x))),const(3))
```

Interpretador

Começaremos pela definição da função.

```
evall(fn(Param, Body), _, fval(Param, Body)).
```

`fval` representa a implementação da função (parâmetro e corpo).

Agora, a aplicação da função.

```
evall(apply(Function, Arg), C, R) :-
    evall(Function, C, fval(Param, Body)),
    evall(Arg, C, ParamValue),
    evall(Body, [bind(Param, ParamValue)|C], R).
```

Legal.. 🤓

```
?- evall(apply(fn(x,times(var(x),var(x))),const(3)),[],X).
X = 9.
```

Um outro exemplo...

```
let val x = 1 in
  let val f = fn n => n + x in
    let val x = 2 in
      f 0
    end
  end
end
```

Qual seria o valor do programa abaixo em ML? 😬
E na nossa linguagem? 😬😬

SML: `val it = 1: int;`

Nossa linguagem:

```
?- eval1(  
|   let(x, const(1),  
|       let(f, fn(n, plus(var(n), var(x))),  
|           let(x, const(2),  
|               apply(var(f), const(0))  
|           )  
|       )  
|   ),  
|   [], x).  
X = 2.
```

O que aconteceu? 😬

Ooops! Implementamos *acidentalmente* um escopo dinâmico 😬.

- Complexa de se implementar de forma eficiente.
- Quebra encapsulamento da função.
- Quase todas linguagens utilizam escopo estático.

O que é preciso para torná-la escopo estático? 😬

Para que haja escopo estático, as variáveis da função devem estar **ligadas ao contexto de sua definição**.

É possível haver escopo estático nesta regra? 😬

```
eval1(fn(Param, Body), _, fval(Param, Body)).
```

Qual mudança devemos fazer? 😬

Solução: incluir na definição da função o contexto do local onde foi criada.

```
eval1(fn(Param, Body), C, fval(Param, Body, C)).
```

```
eval1(apply(Function, Arg), C, R) :- eval1(Function, C, fval(Param, Body, Cfn)),  
    eval1(Arg, C, ParamValue),  
    eval1(Body, [bind(Param, ParamValue)|Cfn], R).
```

Agora sim! 😎

```
?- eval1(  
|   let(x, const(1),  
|       let(f, fn(n, plus(var(n), var(x))),  
|           let(x, const(2),  
|               apply(var(f), const(0))  
|           )  
|       )  
|   ),  
|   [], x).  
X = 2.
```



```
|      )
|      )
|      ),
|  [], x).
X = 1.
```

Para refletir 🤔

1. Quantos parâmetros nossa função suporta?
2. Ela suporta recursividade?
3. Suporta funções de alta-ordem?

Semântica Natural

$$\langle fn(x, E), C \rangle \rightarrow (x, E, C)$$
$$\frac{\langle E_1, C \rangle \rightarrow (x, E_3, C') \quad \langle E_2, C \rangle \rightarrow v_1 \quad \langle E_3, bind(x, v_1) :: C' \rangle \rightarrow v_2}{\langle apply(x, E_1, E_2), C \rangle \rightarrow v_2}$$

Erros

```
?- eval1(apply(const(1),const(1)), [], x).
false.
```

- Que tipo de erro é esse? 🤔
- Em C, ele seria executado em tempo de compilação ou execução? E em Python?

Outras Semânticas

- Existem outras técnicas para representar a semântica de uma linguagem.
- Semântica Natural é apenas uma delas.

Semântica Operacional

- Especifica passo-a-passo o que acontece quando um programa é executado.
- Cada passo representa uma possível computação do programa.
- *Semântica natural* é um tipo de semântica operacional.
- Útil para implementação de linguagens.

Semântica Axiomática

- Descreve o comportamento por um conjunto de asserções.
- Deve ser expressa por `{P} S {Q}`
 - `{P}`: Pré-condições a serem satisfeitas antes da execução de `S`.
 - `S`: Comando a ser executado.
 - `{Q}`: Pós-condições que deverão ser verdadeiras após execução de `S`.
- Útil para provar a corretude de programas implementados.

Já viram isso antes? 🤔^[1]

- Representada por um conjunto de regras.
 - Uma regra mapeia um elemento para um domínio.
 - Regras podem ser compostas em outras mais complexas.
 - Útil para provar propriedades de programas.
-
-

1. [Design by contract](#) ↩